

Radix Sort

The $\Omega(n \log n)$ sorting lower bound does not apply to algorithms that use stronger operations than comparisons. A basic example is [counting sort](#) for sorting integers.

Algorithm 1.18: CountingSort(R)

Input: (Multi)set $R = \{k_1, k_2, \dots, k_n\}$ of integers from the range $[0..\sigma)$.

Output: R in nondecreasing order in array $J[0..n)$.

```
(1) for  $i \leftarrow 0$  to  $\sigma - 1$  do  $C[i] \leftarrow 0$ 
(2) for  $i \leftarrow 1$  to  $n$  do  $C[k_i] \leftarrow C[k_i] + 1$ 
(3)  $sum \leftarrow 0$ 
(4) for  $i \leftarrow 0$  to  $\sigma - 1$  do          // cumulative sums
(5)      $tmp \leftarrow C[i]$ ;  $C[i] \leftarrow sum$ ;  $sum \leftarrow sum + tmp$ 
(6) for  $i \leftarrow 1$  to  $n$  do          // distribute
(7)      $J[C[k_i]] \leftarrow k_i$ ;  $C[k_i] \leftarrow C[k_i] + 1$ 
(8) return  $J$ 
```

- The time complexity is $\mathcal{O}(n + \sigma)$.
- Counting sort is a [stable](#) sorting algorithm, i.e., the relative order of equal elements stays the same.

Similarly, the $\Omega(\sum LCP(\mathcal{R}) + n \log n)$ lower bound does not apply to string sorting algorithms that use stronger operations than symbol comparisons. **Radix sort** is such an algorithm for **integer alphabets**.

Radix sort was developed for sorting large integers, but it treats an integer as a **string of digits**, so it is really a string sorting algorithm.

There are two types of radix sorting:

- MSD radix sort** starts sorting from the beginning of strings (most significant digit).

- LSD radix sort** starts sorting from the end of strings (least significant digit).

```

#include <stdio.h>

int getMax(int arr[], int n)
{
    int mx = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > mx)
            mx = arr[i];
    return mx;
}

void countSort(int arr[], int n, int exp)
{
    int output[n];
    int i, count[10] = { 0 };
    for (i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;
    for (i = 1; i < 10; i++)
        count[i] += count[i - 1];

    // Build the output array
    for (i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }
    for (i = 0; i < n; i++)
        arr[i] = output[i];
}

void radixsort(int arr[], int n)
{
    int m = getMax(arr, n);
    for (int exp = 1; m / exp > 0; exp *= 10)
        countSort(arr, n, exp);
}

void print(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

```

```
int main()
{
    int arr[] = { 170, 45, 75, 90, 802, 24, 2, 66 };
    int n = sizeof(arr) / sizeof(arr[0]);
    radixsort(arr, n);

    print(arr, n);
    return 0;
}
```